



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Divide and Conquer Algorithm Comparison

Course Code: CSA0614

Course Name: Design and Analysis of Algorithms

1

Presented By:

Athavan k(192524036)

Under the supervision:

Dr. A.Sairam



Problem Statement

Algorithm A :

$$T(n) = 2T(n/2) + n$$

Algorithm B :

$$T(n) = 2T(n/2) + n^2$$

Objective:

- Understand Divide and Conquer strategy.
- Apply Master Theorem.
- Determine time complexity.
- Compare recursive structures.
- Identify the more efficient algorithm.



The Divide-and-Conquer

Divide:

Split the problem of size n into 2 subproblems, each of size $n/2$.

Conquer:

Recursively solve each subproblem using the same algorithm.

Combine:

Merge the subproblem results — this step's cost is where the two algorithms differ.

Two Algorithms Under Comparison

ALGORITHM A

Linear-Cost Combine (e.g., Merge Sort style)

$$T(n) = 2T(n/2) + n$$

Splits input into two halves, recurses, then combines results in linear time — e.g., merging two sorted sub-arrays element by element.

ALGORITHM B

Quadratic-Cost Combine (all-pairs style)

$$T(n) = 2T(n/2) + n^2$$

Splits input into two halves, recurses, then combines results by comparing every element pair across the two halves — an $O(n^2)$ combine step.



Pseudocode for Both Algorithms

ALGORITHM A — $T(n) = 2T(n/2) + n$

ALGO_A(arr, low, high):

 if low $\geq$ high: return

 mid = (low + high) / 2

 ALGO_A(arr, low, mid) // $T(n/2)$

 ALGO_A(arr, mid+1, high) // $T(n/2)$

 LINEAR_COMBINE(arr, low, 4
 mid, high) // $O(n)$

ALGORITHM B — $T(n) = 2T(n/2) + n^2$

ALGO_B(arr, low, high):

 if low $\geq$ high: return

 mid = (low + high) / 2

 ALGO_B(arr, low, mid) // $T(n/2)$

 ALGO_B(arr, mid+1, high) // $T(n/2)$

 for i in left_half:

 for j in right_half:

 COMPARE(i, j) // $O(n^2)$



Dry Run Recursion Tree for $n = 8$

Algorithm A: $T(n) = 2T(n/2) + n$

Level	Subproblems	Size	Cost/Level
0	1	8	8
1	2	4	8
2	4	2	8
3	8	1	8

$Total \approx 8 \times (\log_2 8 + 1) = 8 \times 4 \rightarrow \Theta(n \log n)$

Cost is evenly spread across all levels — no single level dominates.

Algorithm B: $T(n) = 2T(n/2) + n^2$

Level	Subproblems	Size	Cost/Level
0	1	8	64
1	2	4	32
2	4	2	16
3	8	1	8

$Total = 64 + 32 + 16 + 8 = 120 \rightarrow \text{root level alone} \approx 53\% \text{ of total cost}$

Cost shrinks geometrically going down the tree — the root (top) level dominates $\rightarrow \Theta(n^2)$.



Applying the Master Theorem

Master Theorem: $T(n) = aT(n/b) + f(n)$ — compare $f(n)$ with $n^{\log_m b}$

Algorithm A — $T(n) = 2T(n/2) + n$

$$a = 2, b = 2, f(n) = n$$

$$n^{\log_2 2} = n^1 = n$$

$$\text{Compare: } f(n) = \Theta(n^{\log_2 2})$$

→ $f(n)$ and $n^{\log_2 2}$ grow at the same rate

→ CASE 2 of Master Theorem applies

$$T(n) = \Theta(n \log n)$$

Algorithm B — $T(n) = 2T(n/2) + n^2$

$$a = 2, b = 2, f(n) = n^2$$

$$n^{\log_2 2} = n^1 = n$$

$$\text{Compare: } f(n) = n^2 = \Omega(n^{1+\epsilon}) \text{ for } \epsilon = 1$$

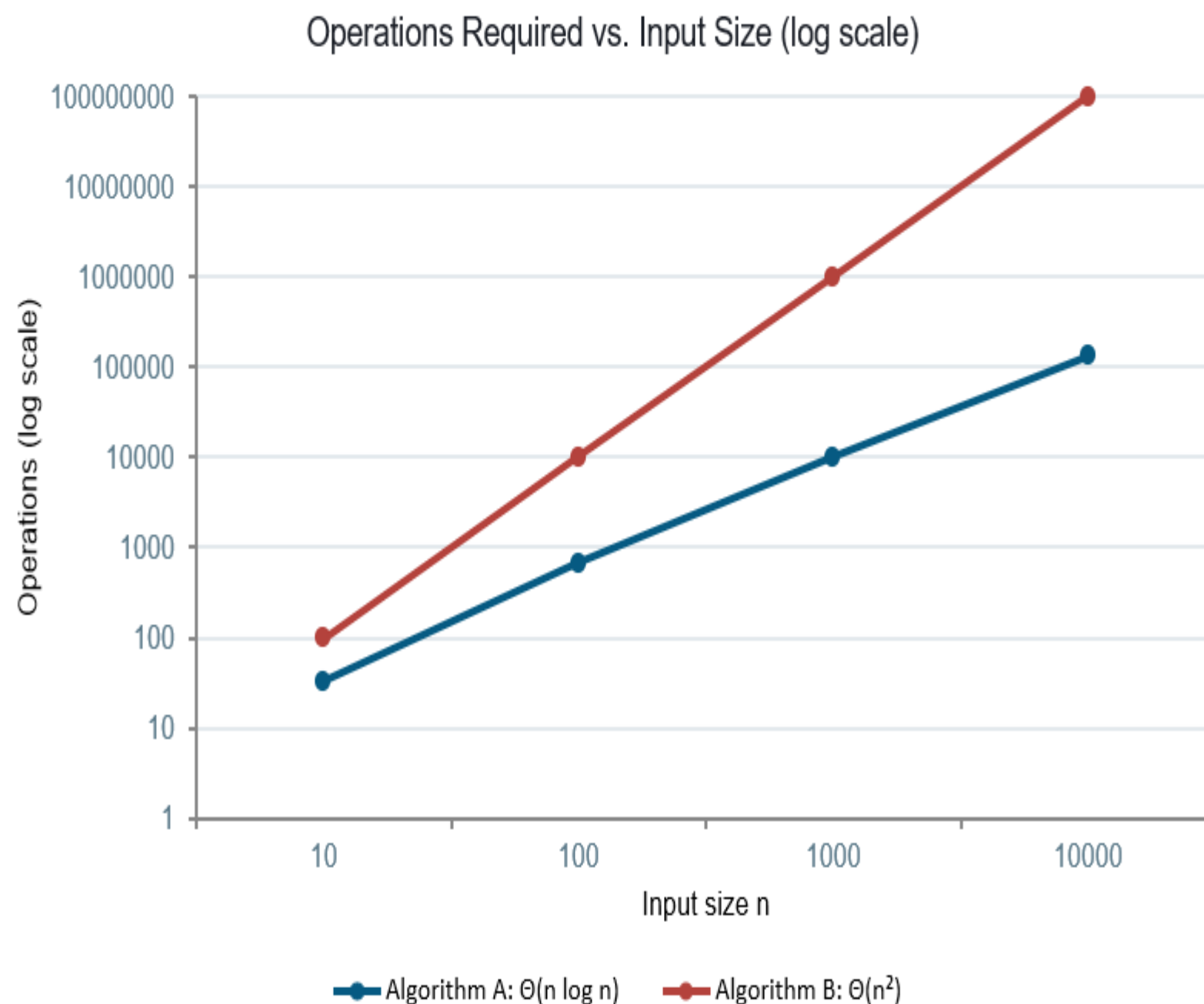
$$\text{Regularity check: } 2 \cdot (n/2)^2 = n^2/2 \leq c \cdot n^2 \text{ for } c = 1/2 < 1 \checkmark$$

→ CASE 3 of Master Theorem applies

$$T(n) = \Theta(n^2)$$



Growth Rate: $\Theta(n \log n)$ vs. $\Theta(n^2)$



Operation Count by Input Size

n	n log n	n ²
10	33	100
100	664	10,000
1000	9,966	1,000,000
10000	132,877	100,000,000

At n = 10,000:

Algorithm A needs $\approx 132,877$ ops

Algorithm B needs $\approx 100,000,000$ ops

Algorithm B requires roughly 750× more operations than Algorithm A at this scale.



Where Each Combine Strategy Is Used

Algorithm A — $\Theta(n \log n)$

- Merge Sort — linear merge of two sorted halves
- External sorting for large disk-resident datasets
- Parallel / distributed sort-and-merge pipelines
- Inversion counting during a linear merge pass

Algorithm B — $\Theta(n^2)$

- Brute-force closest-pair combine across a dividing line
- Naive cross-comparison of two point sets
- Educational baseline before optimizing to $O(n \log n)$
- Small- n subroutines where n^2 combine cost is negligible



Findings and Interpretation

1

The recurring subproblem term is identical

Both algorithms split into $a = 2$ subproblems of size $n/2$ — the difference lies entirely in the combine cost $f(n)$.

2

Combine cost determines the Master Theorem case

A linear $f(n) = n$ keeps pace with the recursion (Case 2); a quadratic $f(n) = n^2$ overwhelms it (Case 3).

3

Root-heavy vs. balanced work distribution

Algorithm B's cost concentrates at the top level of the tree; Algorithm A spreads cost evenly across $\log n$ levels.

4

Algorithm A scales far better for large n

The gap between $\Theta(n \log n)$ and $\Theta(n^2)$ widens rapidly — at $n = 10,000$ the gap is already $\approx 750\times$.



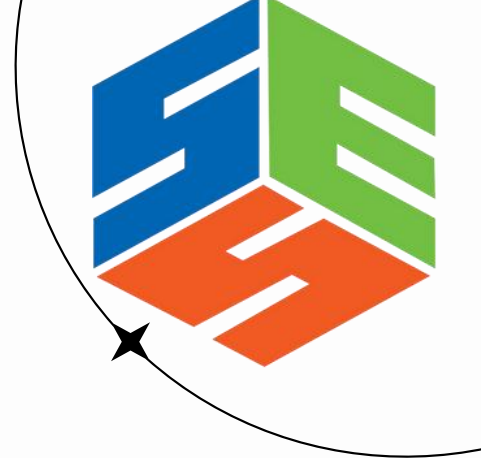
Conclusion

- Although both algorithms divide the problem into the same number of subproblems, the combine cost significantly affects their overall performance.
- Algorithm A is more efficient and scalable because its linear combine cost results in a lower time complexity.
- Algorithm B is less efficient for large input sizes due to its quadratic combine cost.
- Overall, this comparison demonstrates that minimizing the combine cost is essential for designing efficient divide-and-conquer algorithms, making Algorithm A the preferred choice for large-scale applications.

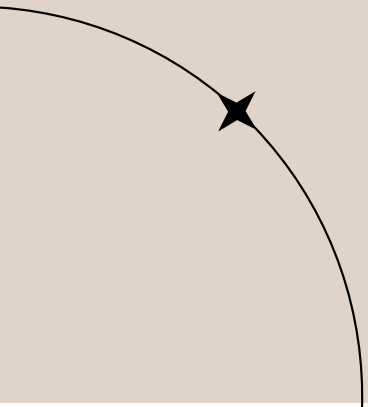
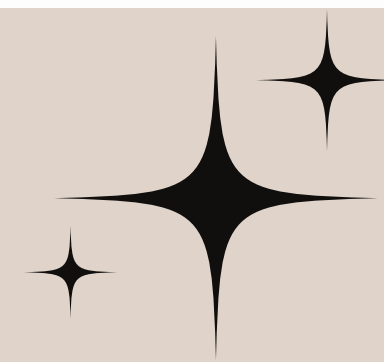


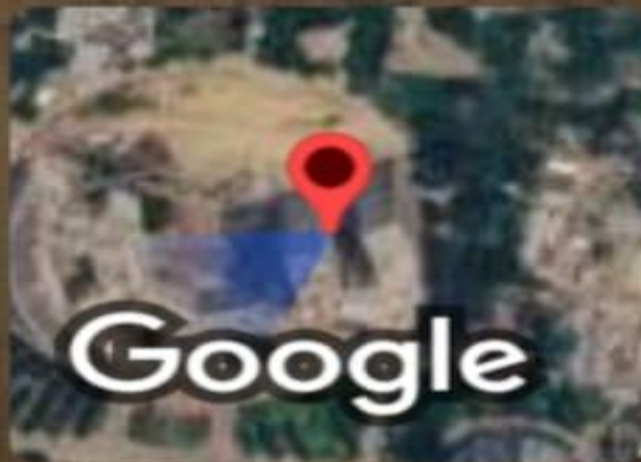
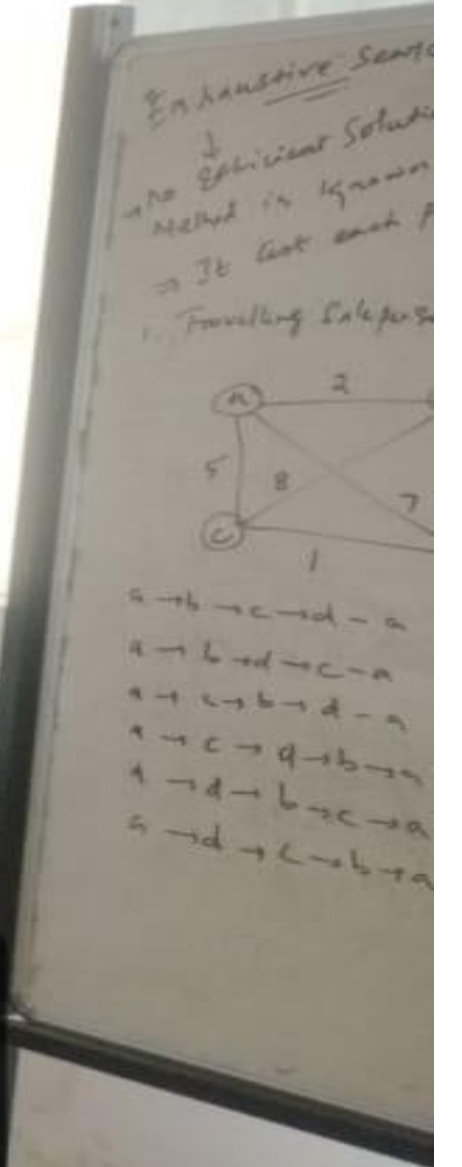
Reference

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- E. Horowitz, S. Sahni, and S. Rajasekaran, Fundamentals of Computer Algorithms, 2nd ed. Hyderabad, India: Universities Press, 2024.
- A. V. Aho, J. E. Hopcroft, and J. D. Ullman, The Design and Analysis of Computer Algorithms. Reading, MA, USA: Addison-Wesley, 1923.
- S. S. Skiena, The Algorithm Design Manual, 3rd ed. Cham, Switzerland: Springer, 2020.
- D. E. Knuth, The Art of Computer Programming, Vol. 1: Fundamental Algorithms, 3rd ed. Boston, MA, USA: Addison-Wesley, 2025.



THANK YOU





Mevalurkuppam, Tamil Nadu, India



22g8+cwh, Mevalurkuppam, Tamil Nadu 602105, India

Lat 13.026496° Long 80.016994°

Friday, 24/07/2026 11:21 AM GMT +05:30

GPS Map Camera